Multi-Arm Bandit

Submitting: Idan Dunsky and Yaniv Kaveh Shtul

Question 1 – Algorithm Design

1a. Why use net rewards (R_i − C_i) instead of raw rewards (R_i)?

The objective is to **maximize net benefit**, not raw output. Each arm has a cost, so the highest-reward arm is not necessarily optimal. For example, an arm with reward 0.9 and cost 0.3 yields net 0.6, while another with reward 0.8 and cost 0.2 also yields net 0.6. Using net reward (R_i − C_i) aligns Q-value estimates with the true objective, ensuring the bandit selects arms that are genuinely profitable after costs.

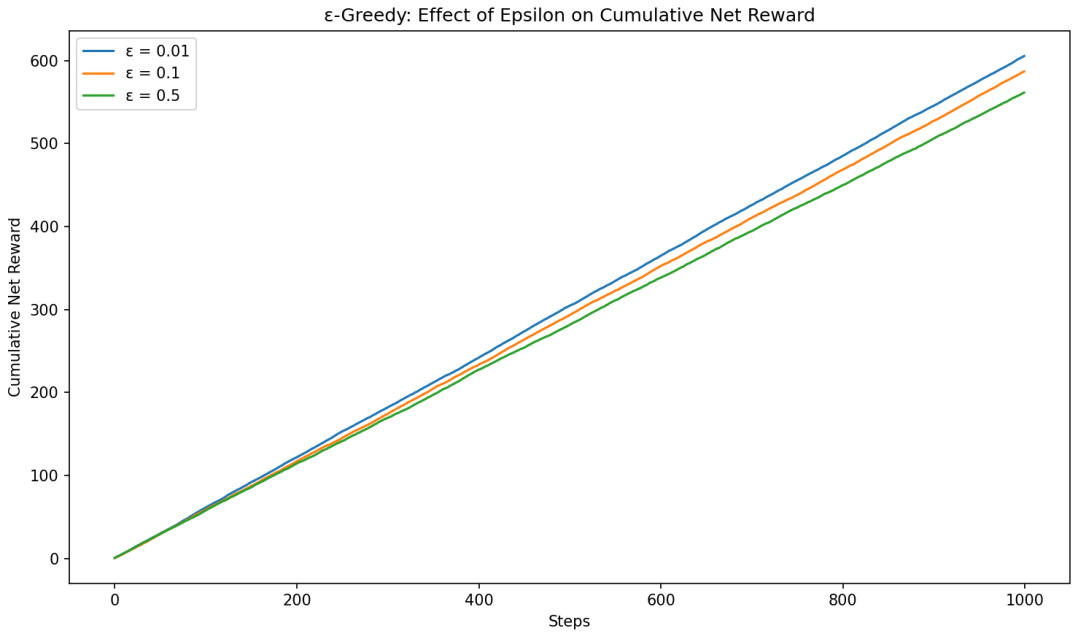
1b. How does ε-Greedy balance exploration and exploitation?

At each step the algorithm flips a biased coin: with probability **ε** it **explores** a uniformly random arm; with probability **1 − ε** it **exploits** the arm with the highest current Q-value. This prevents the agent from being permanently trapped by a lucky early sample from a suboptimal arm.

Too low ε (e.g., 0.01): Almost pure exploitation; if it commits to a suboptimal arm early, it rarely escapes, yielding high regret. **Too high ε (e.g., 0.5):** Wastes half of every step on random exploration even after the best arm is known. **Optimal ε** is problem-dependent; ≈ 0.1 typically balances learning speed and exploitation well.

Question 2 – Experimentation

2a. Effect of Different Epsilon Values (ε = 0.01, 0.1, 0.5)

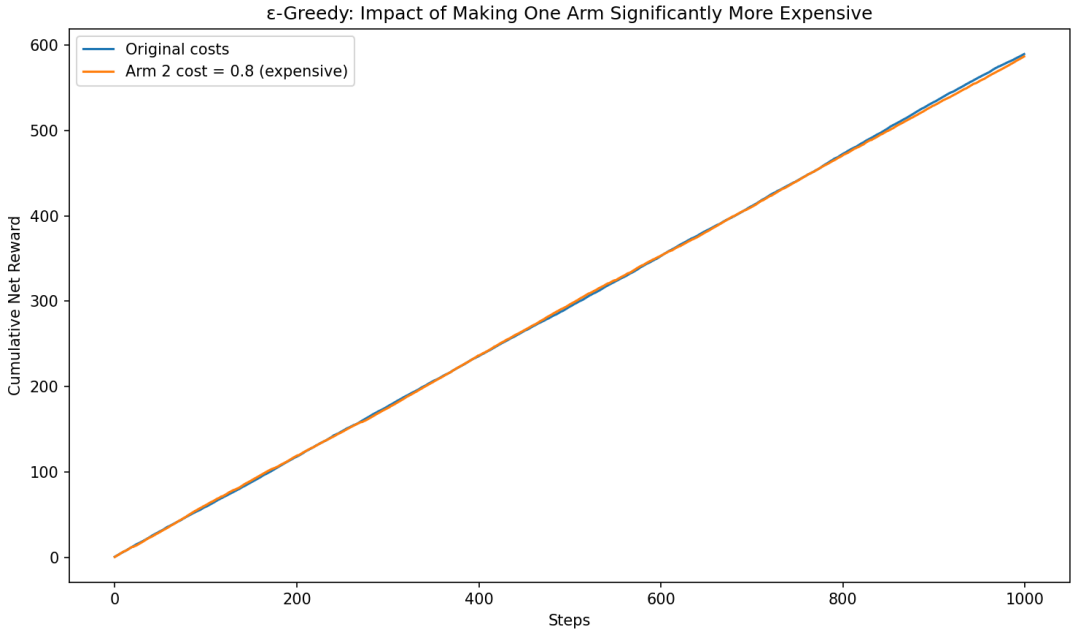


Epsilon comparison

ε = 0.01 starts slow and may commit early to a suboptimal arm; ε = 0.1 achieves the best cumulative net reward — enough exploration to identify the optimal arm, then mostly exploit; ε = 0.5 explores too much throughout, yielding noticeably lower reward. **A moderate ε (≈ 0.1) is best for 1000 steps.**

2b. Modifying Costs — Making One Arm Significantly More Expensive

We raised the cost of arm 2 from 0.3 to 0.8, making its net reward 0.1 (worst of all arms).



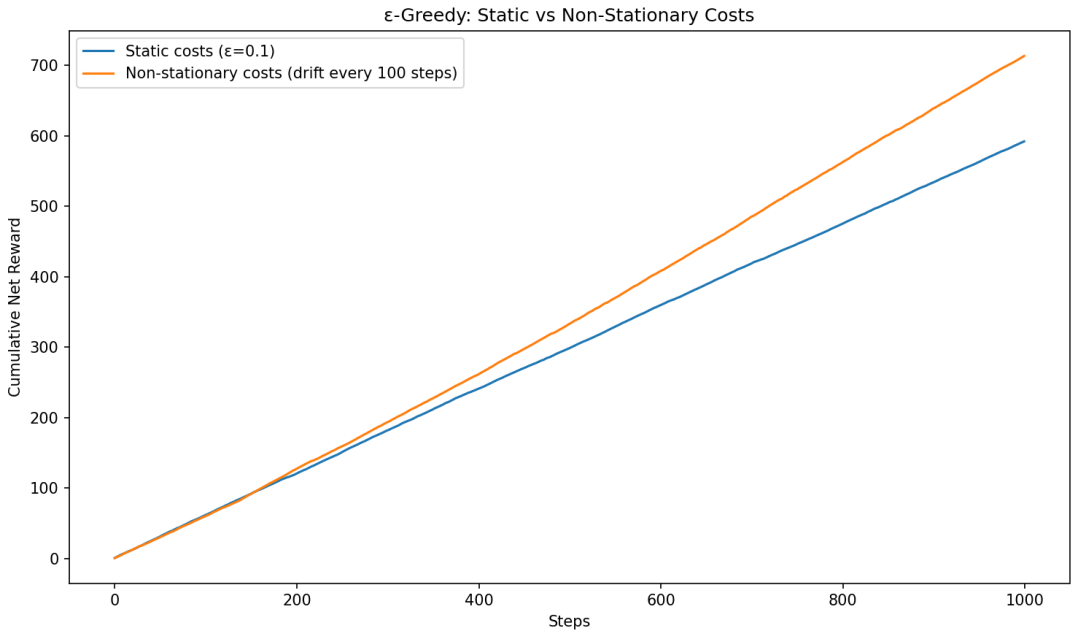
Modified costs

The modified-cost run shows lower cumulative reward early while the agent still samples arm 2, then recovers once it shifts to arm 0 (net 0.6). The Q-value estimates (based on net reward) correctly drive the agent away from the now-expensive arm, with a transient cost during re-learning.

Question 3 – Comparison

3a. Non-Stationary Costs

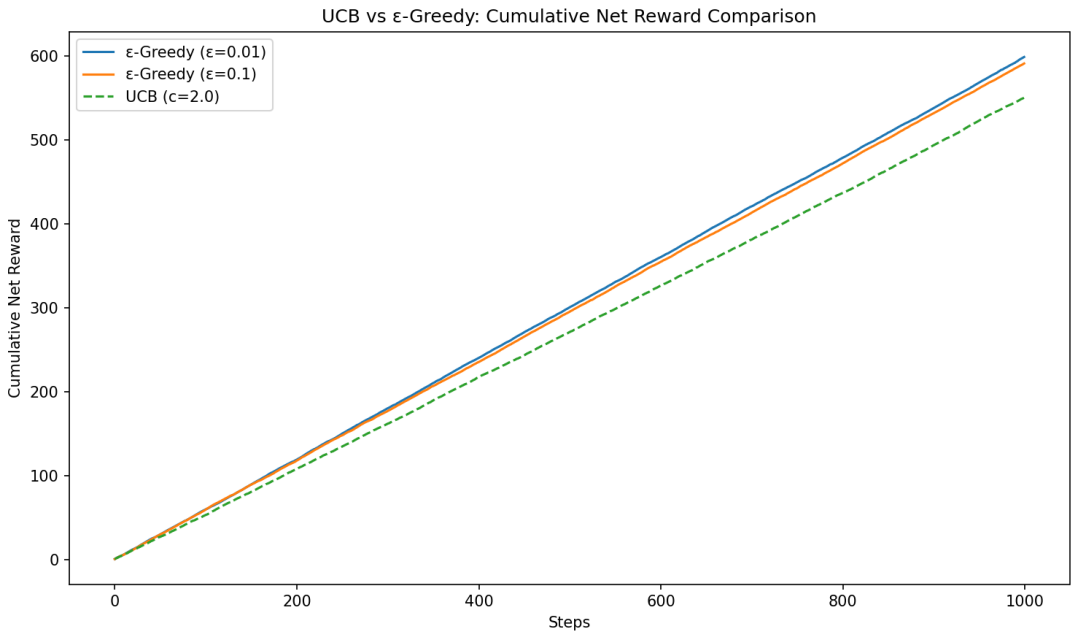
Costs drift by N(0, 0.05) every 100 steps, so the optimal arm can change over time.



Non-stationary costs

The static-cost curve is smooth; the drifting-cost curve shows flatter segments at each shift. Standard ε-greedy adapts only slowly because the 1/N step size makes Q-values increasingly stale. A better approach uses a **constant step-size**: `Q(a) += α * (r - Q(a))` with α ≈ 0.1, giving exponentially decaying weight to older samples so the agent tracks a moving optimum.

3b. UCB vs ε-Greedy

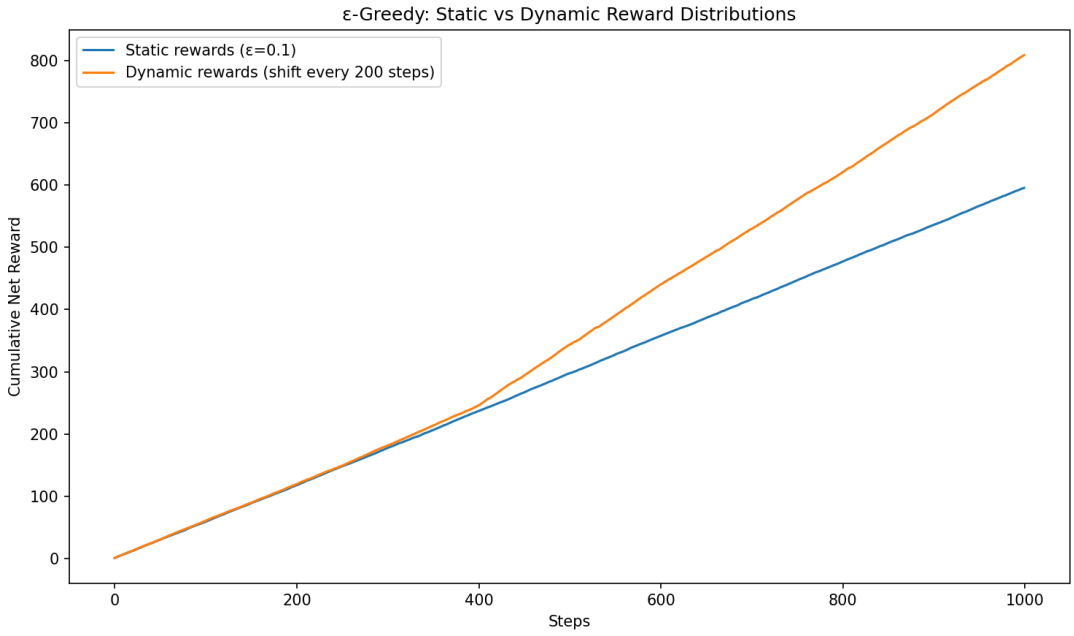


UCB vs ε-Greedy

UCB selects the arm maximising `Q(a) + c * sqrt(ln(t) / N(a))`. The bonus term shrinks as an arm is sampled more, so UCB explores optimistically rather than randomly. Over 1000 steps UCB (c = 2.0) achieves cumulative reward comparable to ε-Greedy but with more principled exploration and provably **O(log T)** regret — no manual ε tuning required. ε-Greedy with very low ε can match UCB in strictly stationary settings, but UCB is generally preferred in practice.

3c. Dynamic Rewards

Mean rewards shift by N(0, 0.2) every 200 steps, changing the optimal arm mid-run.



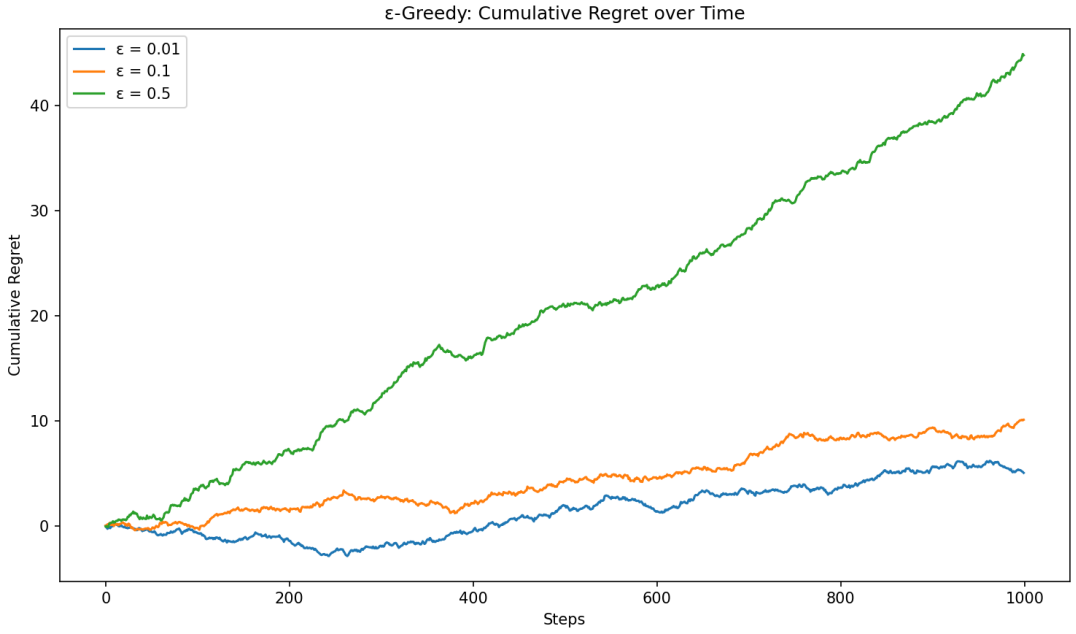
Dynamic rewards

The slope changes at each shift (200, 400, 600, 800) as the agent re-learns. ε-greedy partially adapts via random exploration but its 1/N update means Q-values become stale. **A constant step-size α** lets Q-values track the current distribution.

Question 4 – Analysis

4a. Regret Calculation

Regret at step t is `Regret(t) = sum_{s=1}^{t} [V* - r_s]`, where `V* = max_i (mu_i - C_i) = 0.60` (achieved by arms 0 and 2).



Cumulative regret

4b. Growth of Regret over Time

ε = 0.01 has the lowest regret rate when it locks onto the best arm, but risks linear regret if it commits to a suboptimal one. ε = 0.1 grows approximately linearly (a constant fraction ε of steps are exploratory). ε = 0.5 grows fastest, near-linearly.

For fixed-ε greedy, regret grows linearly in T: `E[Regret(T)] = O(ε * T)`. In contrast, **UCB** and **decayed-ε** (e.g., ε_t = c/t) achieve **sublinear O(log T)** regret, meaning per-step average regret → 0 as T → ∞. For T = 1000 the practical gap is modest, but UCB and decayed-ε remain theoretically superior.